COMSM0037

Text Analytics System

Introduction

This work describes my approach to the required tasks. First, extracting entities from scientific documents abstracts. Second, extracting semantic relations. I explored four different methods to do these subtasks. For extracting entities from scientific abstracts, I used the Conditional Random Fields (CRF) and Hidden Markov Models (HMM) based sequence labelling approach. For the second subtask, I used a simple recurrent neural network (RNN).

Dataset Description

The dataset consists of 500 scientific documents annotated manually for Named entity tags and their start and end locations in the documents. After pre-processing, I got 62475 for the training set and 21133 for the test set.

In this coursework:

- I used BLIO (Beginning, Last, Inside, Out) tagging for labelling the sequences.
- For entity extraction task, the labels were B, L, I and O.
- For relation extraction task, the labels were B/L/I-(Name of Tag) and O. For example, B-PROCESS, I-TASK, L-TASK etc.

Data Pre-processing

The data consists of:

- TXT files - Contains the original text of the article
- ANN files - Contains the annotation start and end location, it is a key phrase type and value.

I performed the following steps to convert the data for sub-task (A) to BILO encoding:

- Loop over each word in the TXT files and search it in their respective ANN file annotations.
- If the word is found in the ANN file annotations, then I assign it:

  B - If the annotation starts with the word
  I - If word lies inside the annotation
  L - If the annotation ends with the word
  O - If the word is not among the annotations

Also, I attached Name of the tag (PROCESS, TASK, MATERIAL) with the BILO tags.

Task 1: extracting entities from scientific abstract

<u>Conditional Random Fields (CRF)</u>

CRF is well suited for the Named entity recognition tasks because it models tagging decisions jointly and can relax the assumption of conditional independence of the observed data often used in generative approaches. It encodes a conditional probability distribution over the set of labels and tries to maximize the regularized log-likelihood function. Additionally, CRF avoids the label bias problem. I used python library CRF Suite for CRF implementation.

The features for the CRF method: I trained CRF with 3-gram and 5-gram training samples. For each word, I use its features combined with the features of its previous and next words, depending upon the n-gram we are using. Then, in the next step we slide the word window by one.

NLTK was used for text pre-processing and PoS-Tagging.

Advantages and Disadvantages of CRF

Advantages

- It is suitable method for NLP tasks such as POS Tagging, Named Entity Recognition
- CRF method considers both left and right neighbouring states when predicting a tag
- CRF Finds the globally optimal sequence hence avoid labelling biases that cause problems for the HMM method.
- Often more accurate than HMM method.

Disadvantages

- CRF normalises over all labellings hence have higher training cost than HMM

Evaluating the CRF Model

I used the F-score to evaluate the CRF Model. F-score conveys balance between Precision and Recall and is defined as: 2*((precision*recall)/(precision+recall))

Precision is defined as the number of True Positives divided by the total number of positive predictions. It is also called the Positive Predictive Value (PPV): Precision=TP/(TP+FP).
Recall is defined as the total number of True Positives divided by the total number of positive class values in the data. It is also called Sensitivity or the True Positive Rate: Recall=TP/(TP+FN).

Hidden Markov Model:

A Hidden Markov Model (HMM) is a five-tuple H= (Σ,Q,q0,A,B) where:

- Σ is a finite observation alphabet;
- Q is a finite set of states;
- q0∈Q is the distinguished initial state;
- A:Q×Q→[0..1] is a probability distribution on state transitions: A(q1, q2) is the probability of a transition to stateq2from stateq1;
- B:Q×Σ→[0..1] is a probability distribution on state symbol emissions: B(q, a) is the probability of observing the symbol a when in state q.

For a tag set “Tags” and a finite set of input tokens “Toks”, it is customary to define a bigram HMM part-of-speech tagger H= (Toks, Tags, , A, B), where the probability functions A and B are estimated from a tagged training corpus.

Advantages and Disadvantages of HMM method

Advantages

HMM method Learns the likelihoods of observations then apply Bayes’ rule to predict tags. Therefore, it makes closed-form maximum likelihood estimates, interpretable and modular. Besides, HMM has a strong statistical foundation with efficient learning algorithms where learning can take place directly from raw sequence data. It allows consistent treatment of insertion and deletion penalties in the form of locally learnable methods and can handle inputs of variable length. They are the most flexible generalization of sequence profiles. It can also perform a wide variety of operations including multiple alignment, data mining and classification, structural analysis, and pattern discovery. It is also easy to combine into libraries.

Disadvantages

- HMM method is only dependent on every state and its corresponding observed object: The sequence labeling, in addition to having a relationship with individual words, also relates to such aspects as the observed sequence length, word context and others.
- The target function and the predicted target function do not match:
  HMM acquires the joint distribution P(Y, X) of the state and the observed sequence, while in the estimation issue, we need a conditional probability P(Y|X).

Evaluating of the HMM Model

I chose to adopt the same way to evaluating CRF Model. So, the indicator of evaluation is F-score. As mentioned in the coursework description, if we use a machine learning method in this step, we should use a different way in the second. I used a different way in tokenization in the HMM model. For tokenization in this model, I used nltk.RegexpParser and for the CRF model I used the test_sents = [[token for token,tag in sent] for sent in test_set].

Results

I tested the two models in the test data, the results are not very good, for the CRF method the results are :

```
F1 score for class Material = 0.20930232558139536
F1 score for class Task = 0.043887147335423204
F1 score for class Process = 0.20916905444126074
Macro-average f1 score = 0.1541195091193598
```

As we see, this method cannot predict the three tags very well, it predicts the type of word with very high percentage but in our case, to predict task or materiel or process it's very hard for a machine learning problem to do it well without more details. So, this method is efficient compared to the HMM model. HMM model must define the probability of each word and its place in the sentence. With automating this process, the probability is very low, so the result is not good. It seems that the tags in this data are difficult to be predicted. Here are the results of the HMM model:

```
F1 score for class Material = 0.028639618138424822
F1 score for class Task = 0.02040816326530612
F1 score for class Process = 0.05417118093174431
Macro-average f1 score = 0.034406320778491754
```

Task 2: extracting semantic relations

In this task, I decided to use two methods to extract the relation between words which is synonyms and hyponyms. The relation synonym means that the two entity phrases refer to the same thing (<entity A> is the same as <entity B>) whereas the hyponym represents the first entity is a more specific word than the second. The second entity is a category or more general term to which the first belongs.

<u>First method: NLTK method</u>

This method is based on taking a word from the input text file and printing the synonyms, definitions and example sentences for the word using WordNet. It separates the synonyms from the synset based on the part-of-speech. For example, synonyms that are verbs and the synonyms that are adjectives are printed separately.

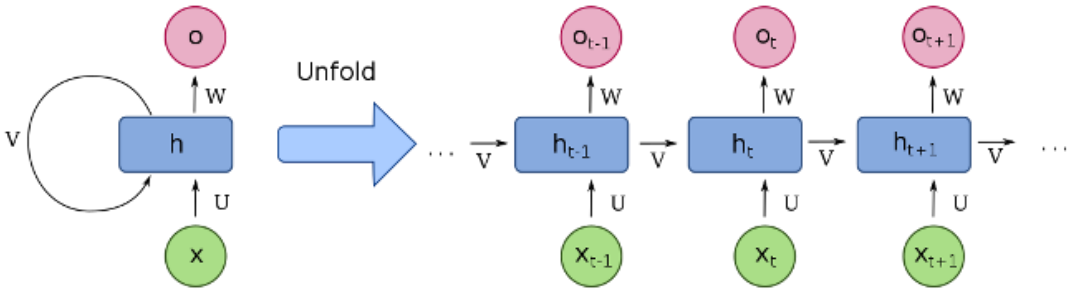
Example for the word flabbergasted the synonyms are 1) flabbergast , boggle , bowl over which are verbs and 2)dumbfounded , dumfounded , flabbergasted , stupefied , thunderstruck , dumbstruck , dumbstricken which are adjectives.

Finally, the method returns the synonym of each word in the same text. This is a basic method that can show us the relation between the words to pass to another complicated method.

Second method: RNN Model

The extraction of relevant information from unstructured documents is a key component in Natural Language Processing (NLP) systems that can be used in many different applications. Neural networks discover features automatically by modelling complex, nonlinear functions in the inputs. Inputs can be either one-hot encodings or fixed word embeddings. One of the limitations here is that interpretability becomes hard and there is also the possibility to overfit and the challenge of integrating linguistic knowledge.

Currently, RNNs are the most popular Deep Learning model to solve the NER task. A simple RNN cell looks like this on the left:



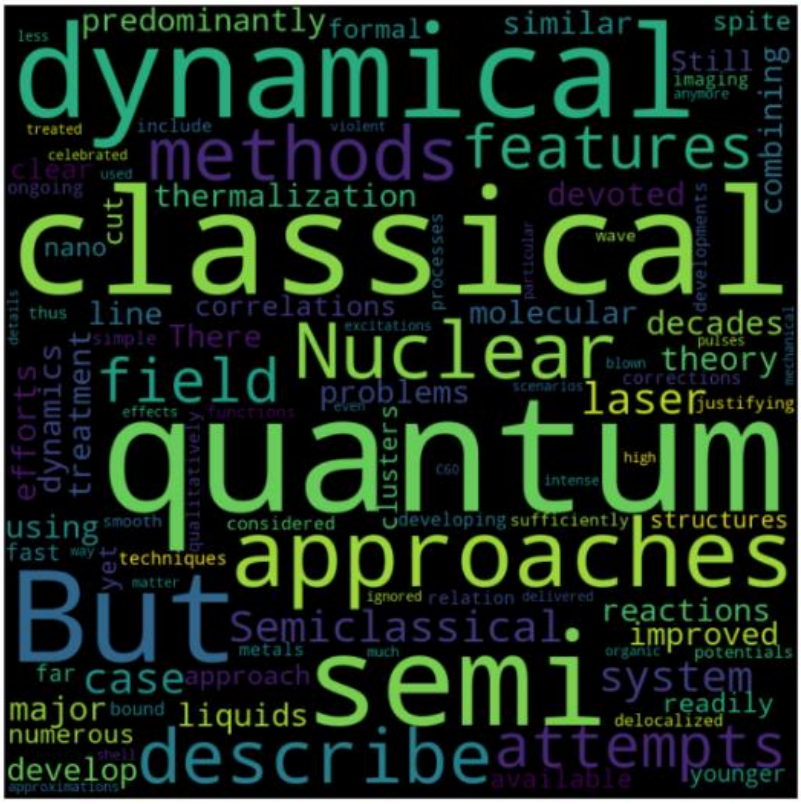
x_i: inputs
h_i: hidden states (memory)
o_i: outputs
U: input weights
V: hidden weights (recurrent connection)
W: output weights

The input is processed sequentially as you might better see on the unfolded representation on the right. Characteristic for RNNs is that all weights/parameters are reused for each iteration step. Only the hidden state is updated dependent on the previous hidden state and the current input. The hidden state works as the memory of the network, that stores the most important information from the previous inputs.

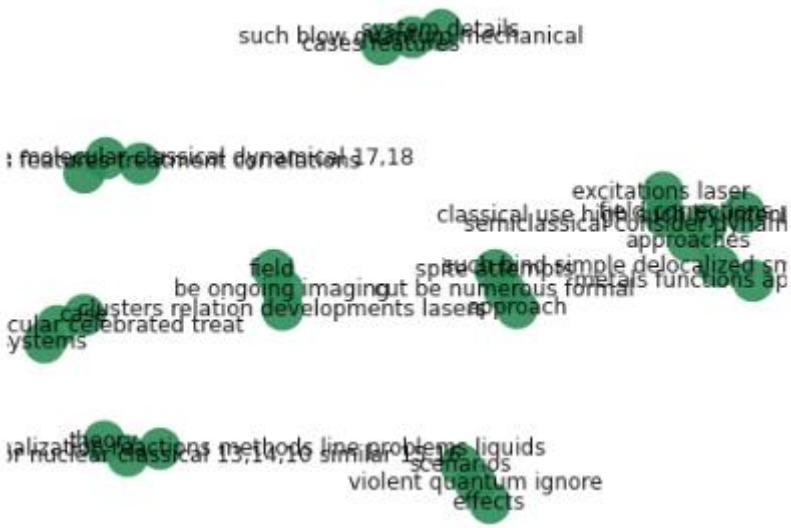
Regarding the confusion matrix of errors, I tested the RNN model with two layers which aim to detect the resemblance between the words and especially to give us the synonym and the hyponym, except that the model could not give a prediction to extract the requested data and this is due to the fact that the training is done on abstracts documents which have few words. Therefore, I have to train the model on the total of the articles to detect the correct information. For this reason, the confusion matrix of errors and the results are not presented. If we want to present this task in a simple and concrete way we can use the nltk library to output the desired information, but this method has some shortcomings, for example, Three-dimensional digital subtraction angiographic and 3D-DSA are synonyms that the nltk does not detect.

Task 3: Exploring the Data

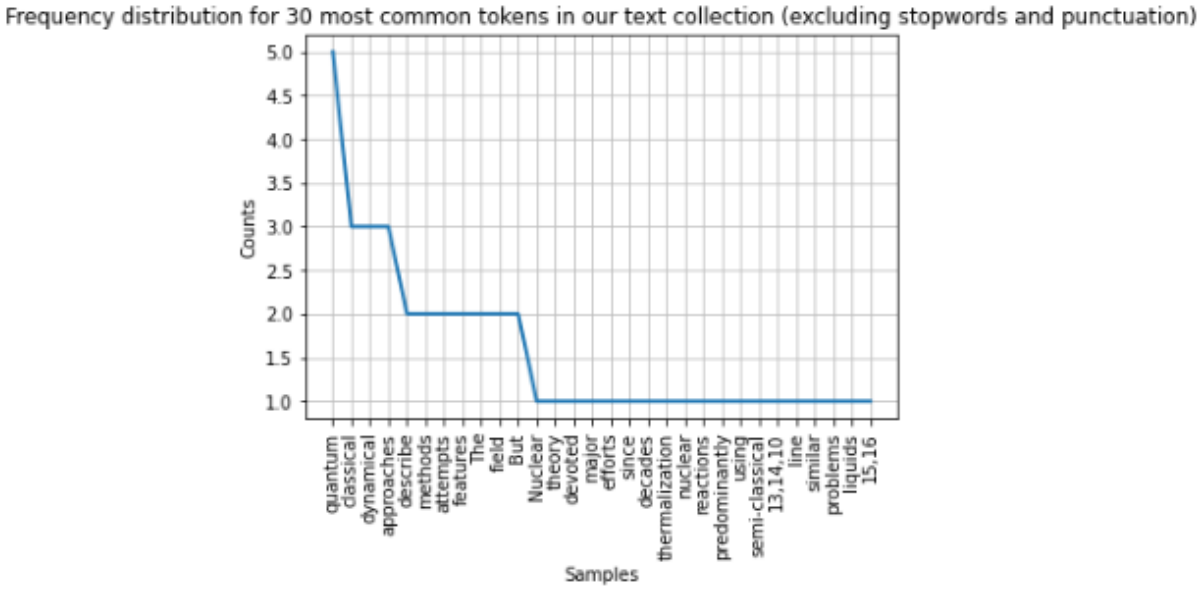
We began by a small descriptive for our data and their contents. Now, we will present some exploratory studies. We start counting the frequency of a word in every text, the figure below shows this in a word cloud figure. This figure can also show the occurrence of each word without number.



Now we will present a graph that shows the relation between the sentences in the text. It shows the relations between some word or a group of words with others in the same text. We do this analysis before we do the second task, to have an idea about the relation between the words.



Now, we draw a graph of word occurrence for the first test in training data (I chose an example). The figure below demonstrates the same results in word cloud plot.



Open Assignment

The paper titled "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList" shows that the current traditional method of evaluating NLP models by splitting the data into train, validation and test sets to estimate the accuracy of the model is not an effective evaluation method for most NLP applications as it overestimates the performance sometimes. Hence, they propose CheckList which is a novel and more efficient methodology for testing NLP models. CheckList works by providing the users with a list of linguistic capabilities, which apply to most NLP tasks and then set up the test as a conceptual matrix with capabilities as rows and test types as columns. By doing this it enables the users to generate various test cases to examine each capability of their model with different test types. The novel aspect of CheckList is that it analyses various natural language capabilities on a given task. Then, it creates test cases to evaluate the model on each of these capabilities. For instance, the Vocabulary and Part of Speech capability relate to if a model has the required vocabulary and their management of the impact of words with different parts of speech on the task. The experimentations used in the paper include using the CheckList tool to test tasks to detect bugs and failures in modern NLP commercial models. One of the tasks was using CheckList to test commercial sentiment analysis models including Microsoft's Text Analytics, Google Cloud's Natural Language and Amazon's Comprehend. They used a dataset of unlabeled airline tweets and created tests for different capabilities and presented a subset with high failure rates. They found that all these models are susceptible to the addition of random shortened URLs, location name changes and person names. Plus, they noticed that none of these models does well with negation as the failure rate is near 100% for all these commercial models when the negation comes at the end of the sentence or between the negation and the sentiment word. Another task is CheckListing Bert and RoB NLP models to detect question pairs that have the same intent. After setting up a selection of tests for Quora Question Pair, they discovered that both models ignore relevant modifiers on the vocabulary. Additionally, both models do not differentiate well with synonyms and antonyms of words and are influenced by typos as well. The models also failed in basic task requirements including NER, Semantic role labelling, Temporal distinctions and differentiating between simple Coreferences. They note that these models do not really recognise named entities impact on questions duplication but rather rely on shortcuts such as anchoring on named entities. Another insightful result is that both models predictions change when the question order is changed. Moreover, they have tested the BERT model by creating a selection of tests for the Machine Comprehension phase. They found that the BERT model fails to identify a couple of things including intensity modifiers, comparisons/superlatives, and Taxonomy such as matching properties like size or colour to adjectives, or comparisons involving antonyms. By using the CheckList approach, they also discovered that the BERT model does not manage short instances with Temporal concepts such as before, after, last, and first, or even with simple examples of Negation in the question or the context. The test cases failure rate indicate that the model does not understand basic Coreferences, subject/object or active/passive distinctions (SRL), which is a piece of important information for machine comprehension. All these failures show how powerful the CheckList tool in measuring NLP models performance. In this research paper, they verified the use of CheckList on models that have been tested extensively before and went through many cycles of bug discovery. They allowed a team responsible for a sentiment analysis model to test their own model using the CheckList methodology. The testing findings were so effective as they tested capabilities they were not considered before and found many previously unknown bugs, which they plan to fix in the next model iteration. Besides, they conducted a user study to further assess the CheckList tool in different circumstances and to check if inexperienced users can gain insights and find flaws in a given model. The user study was conducted on practitioners with little experience in the task. The study enabled the users to generate several tests and the results of the study indicated that the CheckList tool is simple to acquire as users were able to detect significant bugs in a SOTA model in only two hours. Related survey paper titled "Analysis methods in neural language processing: A survey. Transactions of the Association for Computational " explain how important it is to analyse and capture linguistic capabilities through challenging sets such as neural networks for NLP. The main benefit of this approach is to gain systematic control over data. CheckList maintains various of the advantages of challenge sets while mitigating their limitations such as small scale and lack of resemblance to real data.

One of the limitations of the CheckList methodology is that it is only focused on behavioural testing and cannot be used for non-behavioural tests such as data versioning issues, labelling errors, annotator biases, worst-case security issues, or lack of interpretability. When working with this problem in the future they could try to expand their work by providing more test types along with MFT, DIR and INV tests to accurately evaluate and find flaws in NLP models.

Reference

Ribeiro, M.T., Wu, T., Guestrin, C. and Singh, S., 2020. Beyond accuracy: Behavioral testing of NLP models with CheckList. *arXiv preprint arXiv:2005.04118*.

Belinkov, Y. and Glass, J., 2019. Analysis methods in neural language processing: A survey. *Transactions of the Association for Computational Linguistics*, *7*, pp.49-72.

https://www.researchgate.net/publication/324769048_Hierarchical_RNN_for_Information_Extraction_from_Lawsuit_Documents

http://www.inass.org/2020/2020022915.pdf

https://stackabuse.com/python-for-nlp-word-embeddings-for-deep-learning-in-keras/

http://www.nltk.org/api/nltk.tag.html

https://github.com/OC-ScienceIE/scienceie17